
Lista de Exercícios nº2 - Sistemas Embarcados

INSTITUTO FEDERAL DE MINAS GERAIS - Campus Bambuí

Bacharelado em Engenharia de Computação

Instrutor: Williams L. Nicomedes

Aluno: Henrique Augusto G. Fernandes

Disciplina: BiSuEEA.512 - Sistemas Embarcados - 2025

Sinal Analógico

$$v(t) = 1 + \sin(2\pi t) \text{ V}$$

para $0 < t \leq 1$ segundos.

```
In [37]: import numpy as np
import matplotlib.pyplot as plt

# Configuração de estilo dos gráficos
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 10
```

1) Amostragem e Sample-and-Hold

Construção de 10 amostras igualmente espaçadas entre $t = 0$ e $t = 0.9$ segundos.

```
In [38]: # Definição do sinal analógico
def v(t):
    return 1 + np.sin(2 * np.pi * t)

# Parâmetros de amostragem
n_amostras = 10
t_amostras = np.linspace(0, 0.9, n_amostras)
v_amostras = v(t_amostras)

print("Instantes de amostragem (s):")
print(t_amostras)
print("\nValores amostrados (V):")
print(v_amostras)
```

Instantes de amostragem (s):

[0. 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]

Valores amostrados (V):

[1. 1.58778525 1.95105652 1.95105652 1.58778525 1.
 0.41221475 0.04894348 0.04894348 0.41221475]

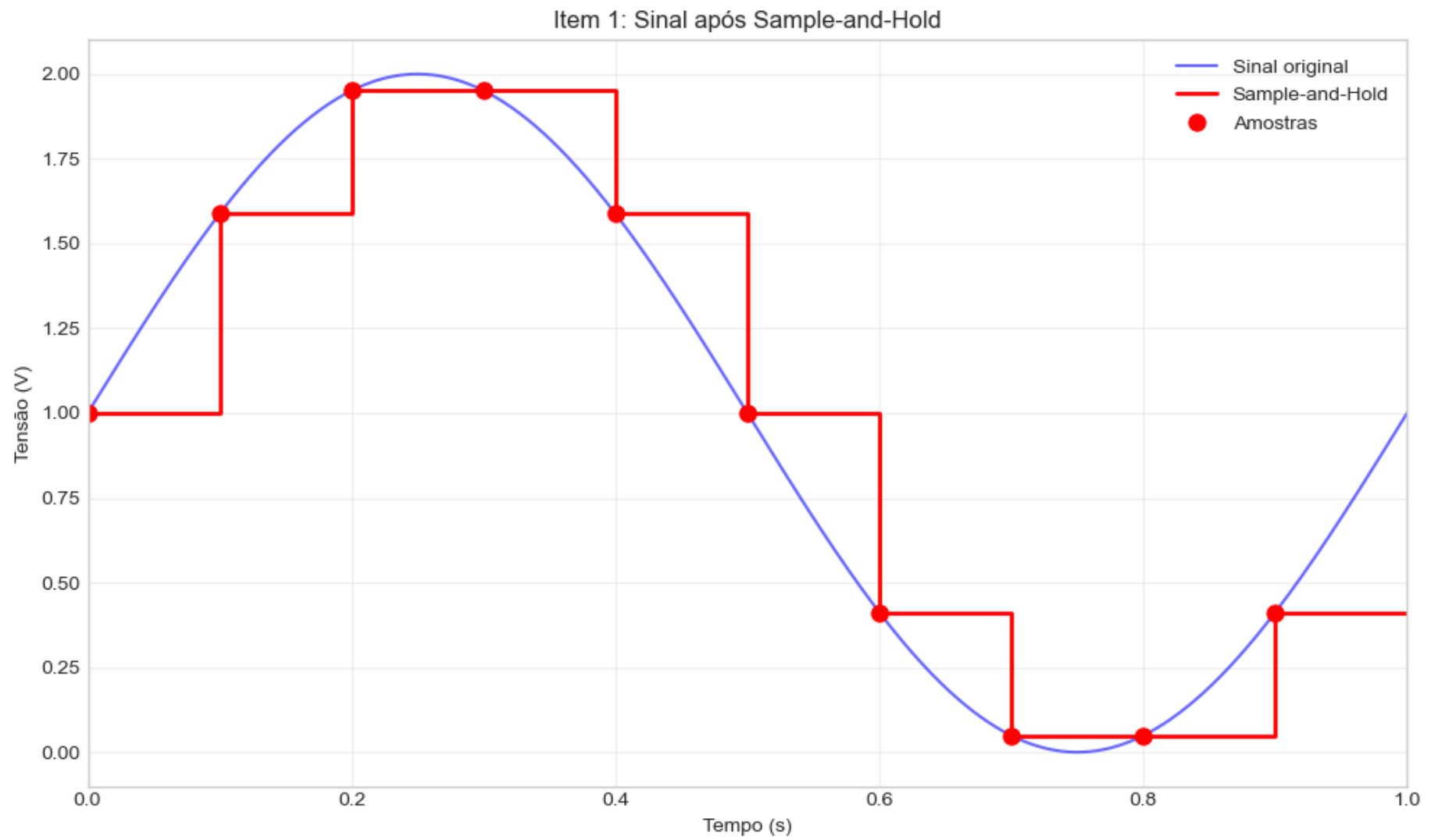
```
In [39]: # Plotagem do sinal Sample-and-Hold
t_continuo = np.linspace(0, 1, 1000)
v_continuo = v(t_continuo)

# Criação do sinal Sample-and-Hold
t_sh = []
v_sh = []

# Loop para construir o sinal Sample-and-Hold conectando cada amostra horizontalmente
for i in range(n_amostras):
    if i < n_amostras - 1:
        # Adiciona ponto inicial e final de cada patamar
        t_sh.extend([t_amostras[i], t_amostras[i+1]])
        v_sh.extend([v_amostras[i], v_amostras[i]])
    else:
        # Último patamar se estende até t = 1.0
        t_sh.extend([t_amostras[i], 1.0])
        v_sh.extend([v_amostras[i], v_amostras[i]])

# Plotagem do sinal Sample-and-Hold
```

```
plt.figure()
plt.plot(t_continuo, v_continuo, 'b-', linewidth=1.5, label='Sinal original', alpha=0.6)
plt.plot(t_sh, v_sh, 'r-', linewidth=2, label='Sample-and-Hold')
plt.plot(t_amostras, v_amostras, 'ro', markersize=8, label='Amostras')
plt.xlabel('Tempo (s)')
plt.ylabel('Tensão (V)')
plt.title('Item 1: Sinal após Sample-and-Hold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```



2) Quantização

Processo de quantização com $N = 4$ bits e $V_{REF} = 2$ V.

```
In [40]: # Parâmetros do conversor ADC
N = 4 # Número de bits
VREF = 2.0 # Tensão de referência (V)
n_niveis = 2**N # Número de níveis de quantização
delta = VREF / n_niveis # Passo de quantização

print(f"Número de bits: {N}")
print(f"Tensão de referência: {VREF} V")
print(f"Número de níveis: {n_niveis}")
print(f"Passo de quantização: {delta:.4f} V")

# Função de quantização
def quantizar(v_in, vref, n_bits):
    n_levels = 2**n_bits
    step = vref / n_levels

    # Limitação aos valores de referência
    v_in = np.clip(v_in, 0, vref)

    # Cálculo do nível de quantização
    nivel = np.floor(v_in / step).astype(int)

    # Ajuste para o nível máximo
    nivel = np.clip(nivel, 0, n_levels - 1)

    # Tensão quantizada (centro do intervalo)
    v_quant = (nivel + 0.5) * step

    return nivel, v_quant

# Quantização das amostras
niveis_quant, v_quantizadas = quantizar(v_amostras, VREF, N)

print("\nNíveis de quantização:")
print(niveis_quant)
print("\nTensões quantizadas (V):")
print(v_quantizadas)
```

Número de bits: 4

Tensão de referência: 2.0 V

Número de níveis: 16

Passo de quantização: 0.1250 V

Níveis de quantização:

[8 12 15 15 12 8 3 0 0 3]

Tensões quantizadas (V):

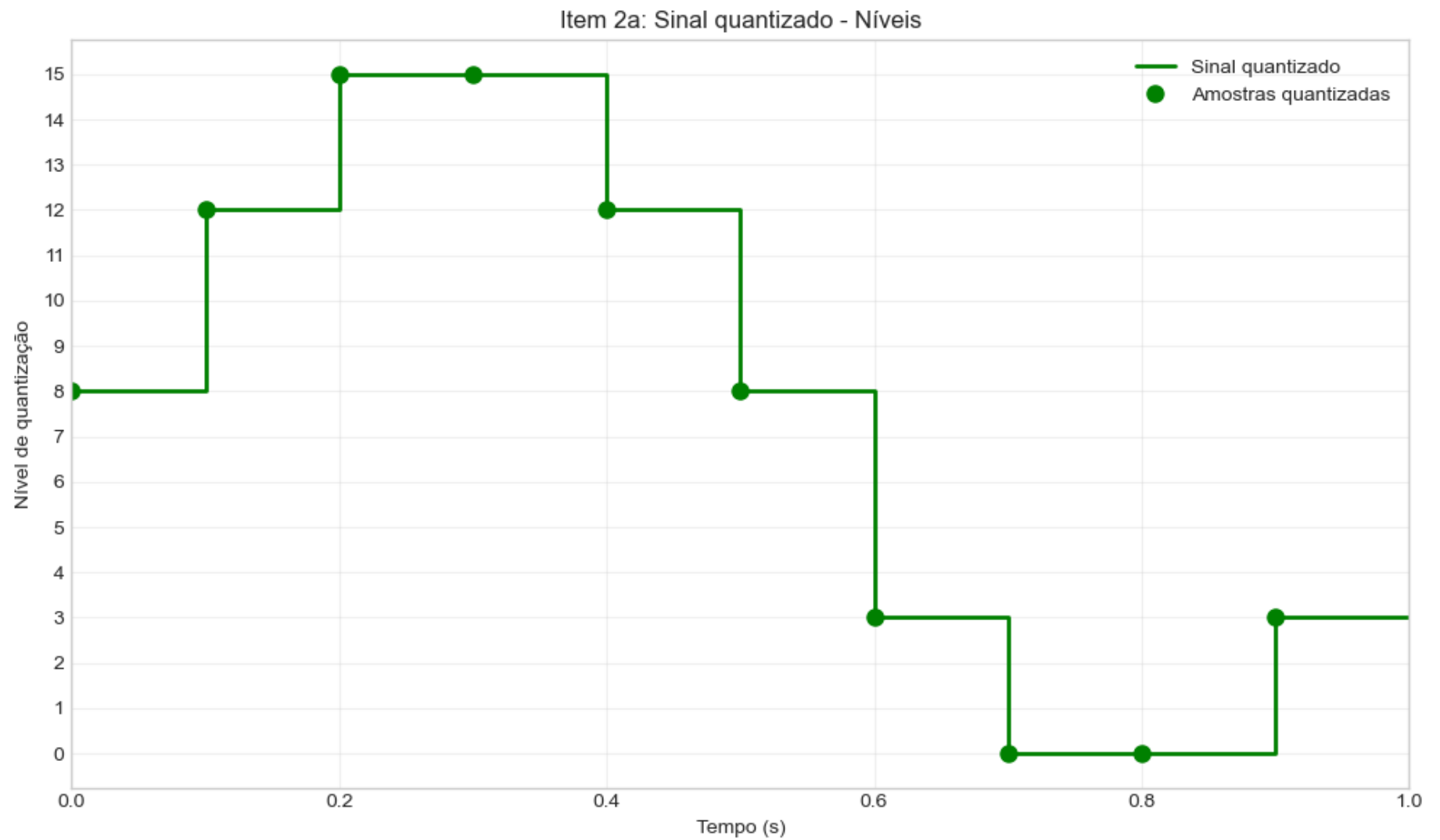
[1.0625 1.5625 1.9375 1.9375 1.5625 1.0625 0.4375 0.0625 0.0625 0.4375]

2. a) Gráfico com níveis de quantização no eixo vertical

```
In [41]: # Criação do sinal quantizado para plotagem contínua
t_quant = []
nivel_quant = []

# Loop para construir o sinal quantizado em níveis
for i in range(n_amostras):
    if i < n_amostras - 1:
        t_quant.extend([t_amostras[i], t_amostras[i+1]])
        nivel_quant.extend([niveis_quant[i], niveis_quant[i]])
    else:
        t_quant.extend([t_amostras[i], 1.0])
        nivel_quant.extend([niveis_quant[i], niveis_quant[i]])

# plotagem do sinal quantizado em níveis
plt.figure()
plt.plot(t_quant, nivel_quant, 'g-', linewidth=2, label='Sinal quantizado')
plt.plot(t_amostras, niveis_quant, 'go', markersize=8, label='Amostras quantizadas')
plt.xlabel('Tempo (s)')
plt.ylabel('Nível de quantização')
plt.title('Item 2a: Sinal quantizado - Níveis')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)
plt.yticks(range(n_niveis))
plt.tight_layout()
plt.show()
```



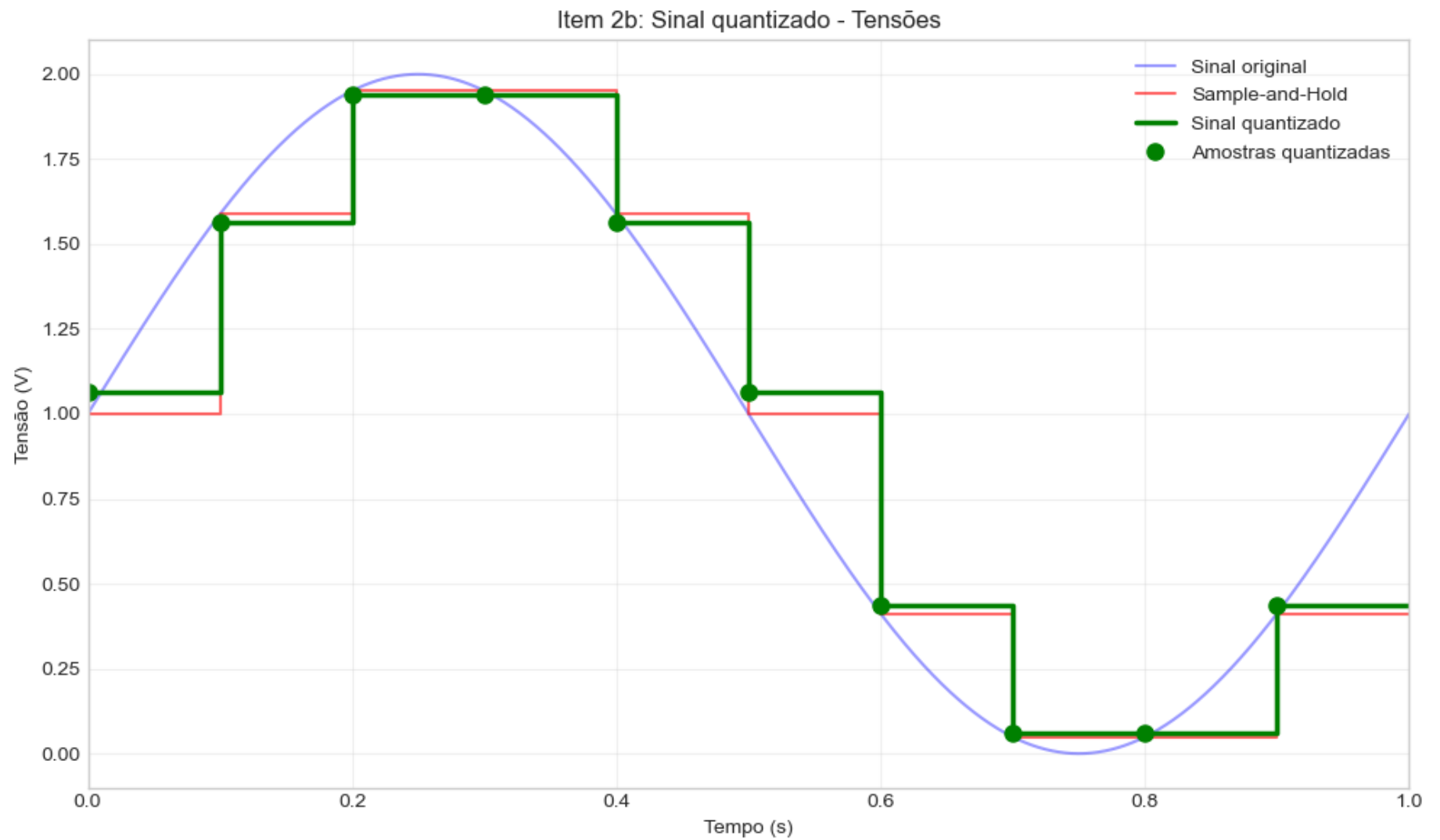
2. b) Gráfico com tensões quantizadas no eixo vertical

```
In [42]: # Criação do sinal quantizado em tensão  
v_quant_plot = []
```



```
# Loop para construir o sinal quantizado em tensão
for i in range(n_amostras):
    if i < n_amostras - 1: # Se não for a última amostra
        v_quant_plot.extend([v_quantizadas[i], v_quantizadas[i]])
    else: # Última amostra
        v_quant_plot.extend([v_quantizadas[i], v_quantizadas[i]])

# Plotagem do sinal quantizado em tensão
plt.figure()
plt.plot(t_continuo, v_continuo, 'b-', linewidth=1.5, label='Sinal original', alpha=0.4)
plt.plot(t_sh, v_sh, 'r-', linewidth=1.5, label='Sample-and-Hold', alpha=0.6)
plt.plot(t_quant, v_quant_plot, 'g-', linewidth=2.5, label='Sinal quantizado')
plt.plot(t_amostras, v_quantizadas, 'go', markersize=8, label='Amostras quantizadas')
plt.xlabel('Tempo (s)')
plt.ylabel('Tensão (V)')
plt.title('Item 2b: Sinal quantizado - Tensões')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```



3) Codificação

Conversão dos níveis de quantização em sequência de bits (bitstream).

```
In [43]: # Função de codificação
def codificar(niveis, n_bits):
    codigos = []
    for nivel in niveis:
        codigo = format(nivel, f'0{n_bits}b')
        codigos.append(codigo)
    return codigos

# Codificação das amostras
codigos_binarios = codificar(niveis_quant, N)

# Criação do bitstream
bitstream = ''.join(codigos_binarios)

print("Codificação das amostras:\n")
print("Amostra | Tempo (s) | Tensão (V) | Nível | Código binário")
print("-" * 65)
for i in range(n_amostras):
    print(f"    {i+1:2d} |    {t_amostras[i]:.3f} |    {v_amostras[i]:.4f} |    {niveis_quant[i]:2d} |    {codigos_binario[i]}")

print(f"\nBitstream completo: {bitstream}")
print(f"Tamanho do bitstream: {len(bitstream)} bits")
```

Codificação das amostras:

Amostra | Tempo (s) | Tensão (V) | Nível | Código binário

1		0.000		1.0000
2		0.100		1.5878
3		0.200		1.9511
4		0.300		1.9511
5		0.400		1.5878
6		0.500		1.0000
7		0.600		0.4122
8		0.700		0.0489
9		0.800		0.0489
10		0.900		0.4122

Bitstream completo: 1000110011111111110010000011000000000011

Tamanho do bitstream: 40 bits

4) Perturbação/Erro

Modificação de um bit no D-ésimo conjunto de bits e análise do impacto.

```
In [44]: # Definição do último dígito do RA
ultimo_digito_RA = 8 # 0050718

# Caso o último dígito seja 0, usar D = 1
D = ultimo_digito_RA if ultimo_digito_RA != 0 else 1

print(f"Último dígito do RA: {ultimo_digito_RA}")
print(f"Valor de D: {D}")
print(f"Amostra a ser modificada: {D}")

# Verificação de validade
if D > n_amostras:
    print(f"\nAVISO: D ({D}) é maior que o número de amostras ({n_amostras}).")
    print(f"Ajustando D para {n_amostras}.")
    D = n_amostras
```

Último dígito do RA: 8

Valor de D: 8

Amostra a ser modificada: 8

```
In [45]: # Cópia dos dados originais
codigos_binarios_modificados = codigos_binarios.copy()
niveis_quant_modificados = niveis_quant.copy()
v_quantizadas_modificadas = v_quantizadas.copy()

# Índice da amostra a ser modificada (D-1 pois índice começa em 0)
idx = D - 1

# Código original
codigo_original = codigos_binarios_modificados[idx]
print(f"\nCódigo original da amostra {D}: {codigo_original}")

# Modificação do bit menos significativo (último bit)
codigo_modificado = list(codigo_original)
```

```

codigo_modificado[-1] = '0' if codigo_modificado[-1] == '1' else '1'
codigo_modificado = ''.join(codigo_modificado)

print(f"Código modificado da amostra {D}: {codigo_modificado}")

# Atualização dos valores
codigos_binarios_modificados[idx] = codigo_modificado
nivel_modificado = int(codigo_modificado, 2)
niveis_quant_modificados[idx] = nivel_modificado
v_quantizadas_modificadas[idx] = (nivel_modificado + 0.5) * delta

print(f"\nNível original: {niveis_quant[idx]}")
print(f"Nível modificado: {nivel_modificado}")
print(f"Tensão original: {v_quantizadas[idx]:.4f} V")
print(f"Tensão modificada: {v_quantizadas_modificadas[idx]:.4f} V")
print(f"Erro absoluto: {abs(v_quantizadas_modificadas[idx] - v_quantizadas[idx]):.4f} V")

```

Código original da amostra 8: 0000
Código modificado da amostra 8: 0001

Nível original: 0
Nível modificado: 1
Tensão original: 0.0625 V
Tensão modificada: 0.1875 V
Erro absoluto: 0.1250 V

```

In [46]: # Criação do sinal perturbado
v_quant_pert = []

# Loop para construir o sinal quantizado modificado
for i in range(n_amostras):
    if i < n_amostras - 1:
        v_quant_pert.extend([v_quantizadas_modificadas[i], v_quantizadas_modificadas[i]])
    else:
        v_quant_pert.extend([v_quantizadas_modificadas[i], v_quantizadas_modificadas[i]])

# Plotagem comparativa
plt.figure(figsize=(12, 6))

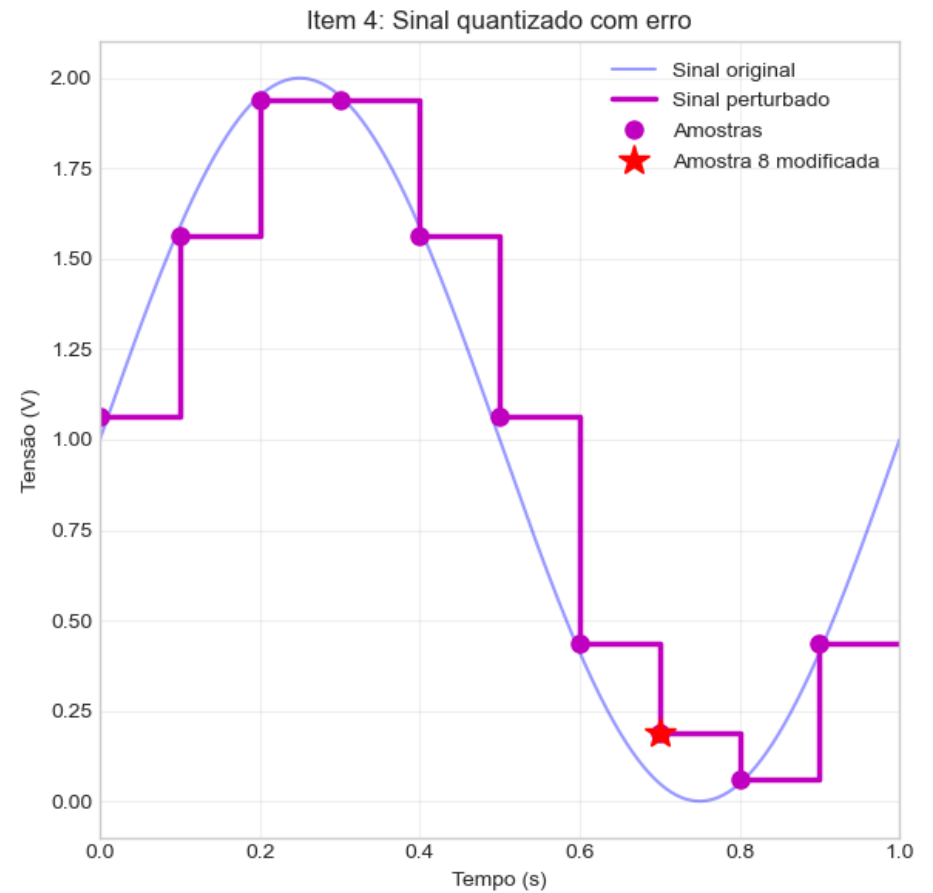
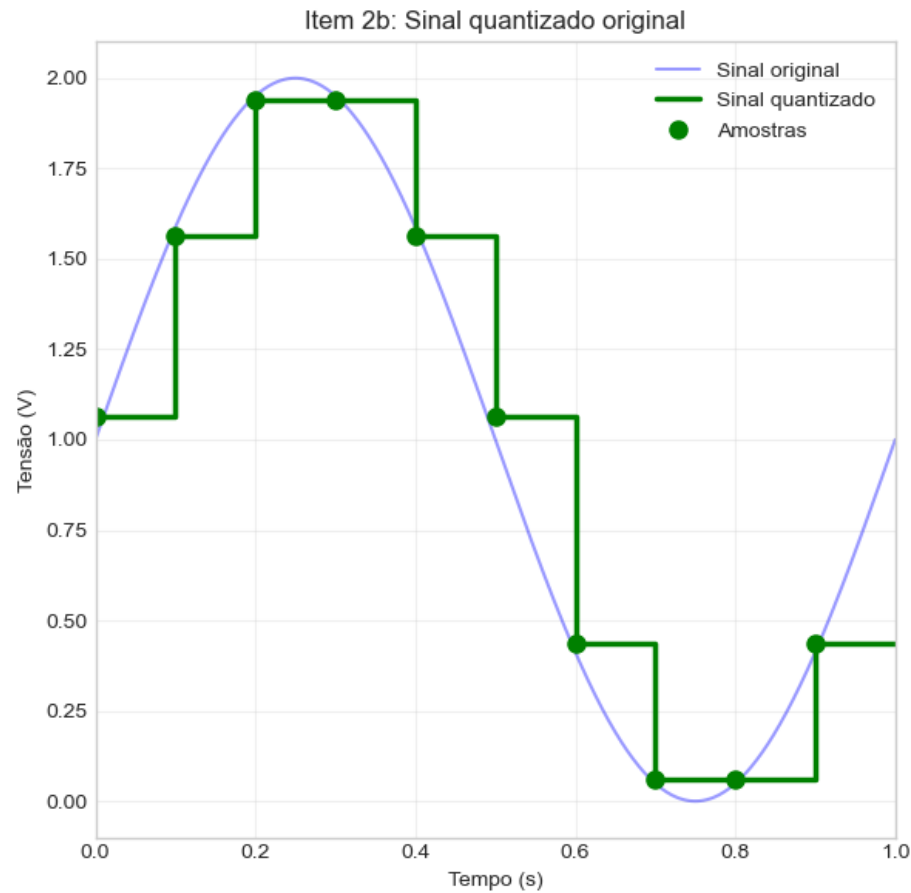
# Sinal original quantizado
plt.subplot(1, 2, 1)

```

```
plt.plot(t_continuo, v_continuo, 'b-', linewidth=1.5, label='Sinal original', alpha=0.4)
plt.plot(t_quant, v_quant_plot, 'g-', linewidth=2.5, label='Sinal quantizado')
plt.plot(t_amostras, v_quantizadas, 'go', markersize=8, label='Amostras')
plt.xlabel('Tempo (s)')
plt.ylabel('Tensão (V)')
plt.title('Item 2b: Sinal quantizado original')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)

# Sinal perturbado
plt.subplot(1, 2, 2)
plt.plot(t_continuo, v_continuo, 'b-', linewidth=1.5, label='Sinal original', alpha=0.4)
plt.plot(t_quant, v_quant_pert, 'm-', linewidth=2.5, label='Sinal perturbado')
plt.plot(t_amostras, v_quantizadas_modificadas, 'mo', markersize=8, label='Amostras')
plt.plot(t_amostras[idx], v_quantizadas_modificadas[idx], 'r*', markersize=15,
         label=f'Amostra {D} modificada')
plt.xlabel('Tempo (s)')
plt.ylabel('Tensão (V)')
plt.title('Item 4: Sinal quantizado com erro')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)

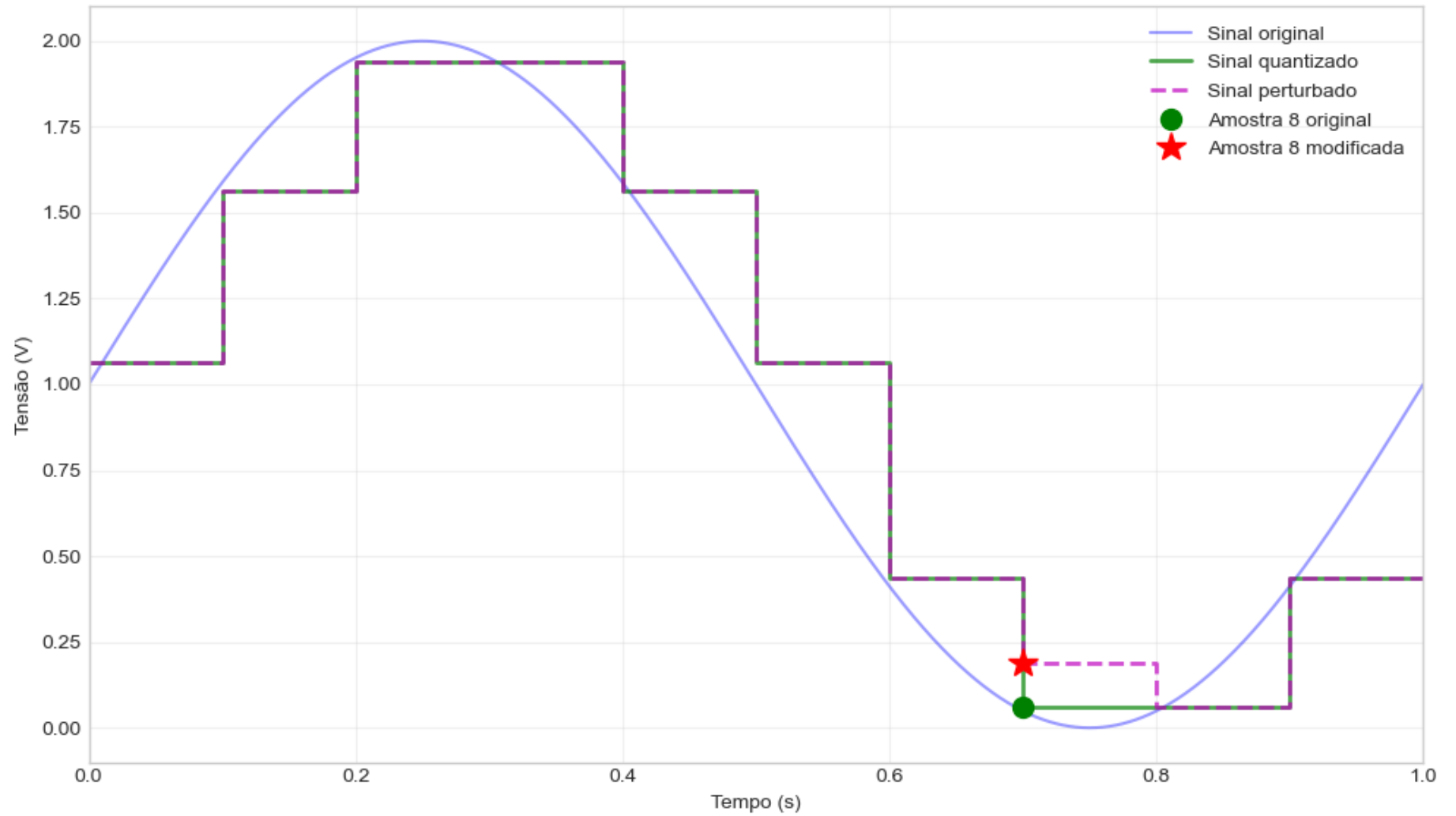
plt.tight_layout()
plt.show()
```



```
In [47]: # Gráfico sobreposto para melhor comparação
plt.figure()
plt.plot(t_continuo, v_continuo, 'b-', linewidth=1.5, label='Sinal original', alpha=0.4)
plt.plot(t_quant, v_quant_plot, 'g-', linewidth=2, label='Sinal quantizado', alpha=0.7)
plt.plot(t_quant, v_quant_pert, 'm--', linewidth=2, label='Sinal perturbado', alpha=0.7)
plt.plot(t_amstras[idx], v_quantizadas[idx], 'go', markersize=10,
         label=f'Amostra {D} original')
plt.plot(t_amstras[idx], v_quantizadas_modificadas[idx], 'r*', markersize=15,
         label=f'Amostra {D} modificada')
plt.xlabel('Tempo (s)')
plt.ylabel('Tensão (V)')
plt.title('Item 4: Comparação entre sinal original e perturbado')
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```

Item 4: Comparação entre sinal original e perturbado



Resumo dos Resultados

```
In [48]: print("="*70)
print("RESUMO DOS RESULTADOS")
print("="*70)

print("\n1. PARÂMETROS DO SISTEMA")
print("-"*70)
print(f"    Sinal:  $v(t) = 1 + \sin(2\pi t)$  V")
print(f"    Número de amostras: {n_amostras}")
print(f"    Taxa de amostragem: {n_amostras/0.9:.2f} amostras/segundo")
print(f"    Resolução ADC: {N} bits")
print(f"    Tensão de referência: {VREF} V")
print(f"    Passo de quantização: {delta:.4f} V")
print(f"    Número de níveis: {n_niveis}")

print("\n2. ERRO DE QUANTIZAÇÃO")
print("-"*70)
erros_quant = v_amostras - v_quantizadas
print(f"    Erro máximo: {np.max(np.abs(erros_quant)):.4f} V")
print(f"    Erro médio: {np.mean(np.abs(erros_quant)):.4f} V")
print(f"    Erro RMS: {np.sqrt(np.mean(erros_quant**2)):.4f} V")

print("\n3. PERTURBAÇÃO INTRODUZIDA")
print("-"*70)
print(f"    Amostra modificada: {D}")
print(f"    Bit alterado: LSB (bit menos significativo)")
print(f"    Erro introduzido: {abs(v_quantizadas_modificadas[idx] - v_quantizadas[idx]):.4f} V")
print(f"    Erro relativo: {abs(v_quantizadas_modificadas[idx] - v_quantizadas[idx])/v_quantizadas[idx]*100:.2f}%")

print("\n4. BITSTREAM")
print("-"*70)
print(f"    Original: {bitstream}")
bitstream_modificado = ''.join(codigos_binarios_modificados)
print(f"    Modificado: {bitstream_modificado}")
print("="*70)
```

```
=====
RESUMO DOS RESULTADOS
=====
```

```
1. PARÂMETROS DO SISTEMA
-----
```

```
Sinal:  $v(t) = 1 + \sin(2\pi t)$  V
Número de amostras: 10
Taxa de amostragem: 11.11 amostras/segundo
Resolução ADC: 4 bits
Tensão de referência: 2.0 V
Passo de quantização: 0.1250 V
Número de níveis: 16
```

```
2. ERRO DE QUANTIZAÇÃO
-----
```

```
Erro máximo: 0.0625 V
Erro médio: 0.0280 V
Erro RMS: 0.0333 V
```

```
3. PERTURBAÇÃO INTRODUZIDA
-----
```

```
Amostra modificada: 8
Bit alterado: LSB (bit menos significativo)
Erro introduzido: 0.1250 V
Erro relativo: 200.00%
```

```
4. BITSTREAM
-----
```

```
Original: 1000110011111111110010000011000000000011
Modificado: 1000110011111111110010000011000100000011
```

```
=====
```